Unité – Discipline - Travail

ECOLE NATIONALE SUPERIEURE DE STATISTIQUE ET D'ECONOMIE APPLIQUEE

PROJET WEBSCRAPING

Analyse de la Mode en Côte d'Ivoire

Réalisé par :
OUEDRAOGO Issa
TRAORE Clayéré Abdoulaye

ENSEIGNANT :
Mr KONATE

AVRIL 2026

sommaire

1. Introduction et contexte 3
1.1 Objectifs du projet 3
1.2 Composition du groupe 3
2. Cadre éthique et légal 4
2.1 Chemins interdits 4
2.2 Mesures de respect éthique mises en place 4
3. Architecture du projet 5
3.1 Services Docker déployés 5
4. Étape 1 Scraping et nettoyage des données (Jalon 1) 6
4.1 Développement du spider (spider.py) 6
4.2 Débogage et corrections du spider 6
4.3 Module de nettoyage (cleaner.py) 6
4.4 Résultats du scraping 7
4.5 Tests pytest (27 tests 100% réussis) 7
5. Étape 2 API Flask + Base de données PostgreSQL 8
5.1 Modèles SQLAlchemy (models.py) 8
5.2 Endpoints REST + Swagger (routes.py) 8
5.3 Documentation Swagger 8
6. Étape 3 Dockerisation 9
6.1 Dockerfile 9
6.2 docker-compose.yml 9
6.3 Difficultés liées à Docker 9
7. Étape 4 Celery et automatisation 10
7.1 Configuration Celery (celery_app.py) 10
7.2 Planification automatique Celery Beat 10
7.3 Suivi des tâches 10
8. Étape 5 Monitoring (Prometheus + Grafana) 11
8.1 Métriques Prometheus 11
8.2 Dashboard Grafana 11
9. Résultats et bilan final 12
9.1 Statistiques clés des données collectées 12
10. Difficultés rencontrées et solutions apportées 13
Conclusion 14

1. Introduction et contexte

Ce rapport présente le projet de web scraping réalisé dans le cadre du cours AS Data Science de l'ENSEA (École Nationale Supérieure de Statistique et d'Économie Appliquée). Le thème retenu est l'analyse du marché de la mode en Côte d'Ivoire à travers les annonces publiées sur la plateforme de e-commerce jiji.co.ci.

Jiji.co.ci est la principale plateforme d'annonces en ligne en Côte d'Ivoire, regroupant des milliers d'articles de mode : vêtements, chaussures, sacs, accessoires, pour femmes, hommes et enfants. Cette source de données constitue un terrain d'analyse pertinent pour comprendre les dynamiques du marché vestimentaire ivoirien, les fourchettes de prix, les zones géographiques de vente et les tendances de catégories.

1.1 Objectifs du projet

- Collecter automatiquement les annonces mode de Jiji.co.ci (max. 500 items selon la charte éthique)
- Nettoyer et structurer les données brutes en un dataset exploitable
- Exposer les données via une API REST documentée avec Swagger
- Automatiser le scraping périodique avec Celery
- Mettre en place un système de monitoring avec Prometheus et Grafana
- Containeriser l'ensemble via Docker

1.2 Composition du groupe

Le projet a été réalisé par 2 étudiants (OUEDRAOGO ISSA & TRAORE Clayéré Abdoulaye), partis de zéro, avec pour objectif d'atteindre le niveau Or (20/20). Le dépôt GitHub est disponible à l'adresse : https://github.com/issa-tech/Webscraping-pipeline-OUEDRAOGO_TRAORE

2. Cadre éthique et légal

Avant tout développement, une vérification du fichier robots.txt de Jiji.co.ci a été effectuée à l'adresse https://jiji.co.ci/robots.txt. Cette vérification a confirmé que les pages de listings mode sont toutes accessibles au scraping.

2.1 Chemins interdits

- /test/*
- /admin/*
- /crm/*
- /auth/facebook*

2.2 Mesures de respect éthique mises en place

1. User-Agent identifié comme projet académique ENSEA dans l'en-tête X-Scraper-Info
2. Délai aléatoire entre requêtes de 1 à 2,5 secondes pour ne pas surcharger le serveur
3. Volume maximum limité à 500 items conformément à la charte éthique du projet
4. Gestion des erreurs HTTP avec 3 tentatives maximum avant abandon
5. Dédoublonnage des données pour éviter toute redondance

Un email de validation a été envoyé au responsable pédagogique (konatengolo@ufhb.edu.ci) avec la justification du choix du site et la capture du robots.txt.

3. Architecture du projet

Le projet est structuré en modules indépendants pour faciliter la maintenabilité et les tests. Voici l'arborescence complète mise en place :

```
webscraping-pipeline/
├── scraper/          → spider.py + cleaner.py
├── api/              → app.py, models.py, routes.py, __init__.py
├── tasks/            → celery_app.py + scrape_task.py
├── monitoring/       → prometheus.yml + grafana/
├── tests/            → test_scraper.py (27 tests pytest)
├── docker-compose.yml
├── Dockerfile
├── run.py
├── requirements.txt
├── .env.example
└── README.md
```

3.1 Services Docker déployés

Critère / Composant	Statut
jiji_api	API Flask + endpoints REST + Swagger
jiji_db	PostgreSQL 15 — base de données principale
jiji_redis	Redis 7 — broker de messages Celery
jiji_celery_worker	Exécution asynchrone des tâches de scraping
jiji_celery_beat	Planificateur — scraping automatique toutes les 6h
jiji_prometheus	Collecte des métriques de l'API
jiji_grafana	Visualisation des métriques en temps réel

4. Étape 1 Scraping et nettoyage des données (Jalon 1)

4.1 Développement du spider (spider.py)

Le spider a été développé avec les bibliothèques requests et BeautifulSoup4. Il cible 4 catégories de mode sur Jiji.co.ci :

- https://jiji.co.ci/womens-fashion — Mode Femme
- https://jiji.co.ci/mens-fashion — Mode Homme
- https://jiji.co.ci/kids-fashion — Mode Bébé & Enfant
- https://jiji.co.ci/fashion-and-beauty — Mode général

Pour chaque annonce, les données suivantes sont extraites :

- Titre de l'annonce
- Prix brut (ex. : CFA 16,000)
- Localisation (ville / quartier)
- URL de l'annonce
- URL de l'image principale
- Date de scraping (scraped_at)
- Catégorie (mode) et source (jiji.co.ci)

4.2 Débogage et corrections du spider

Plusieurs erreurs ont été rencontrées et corrigées lors du développement :

Critère / Composant	Statut
UnicodeEncodeError (latin-1)	Remplacement du tiret long (—) par un tiret simple (-) dans le User-Agent
Erreur HTTP 404	Correction de l'URL /ci/fashion en /fashion-and-beauty
Blocage 403	Remplacement du User-Agent ENSEA par un User-Agent navigateur réel
0 annonce collectée	Correction de l'URL de base du scraper

4.3 Module de nettoyage (cleaner.py)

Le module de nettoyage utilise pandas pour transformer les données brutes en un dataset propre et structuré. Les 4 fonctions principales sont :

clean_price()

Extrait la valeur numérique du prix et la devise. Correction appliquée : la virgule dans 'CFA 16,000' était interprétée comme un séparateur décimal, ce qui donnait 16.0 au lieu de 16 000. La correction consiste à supprimer tous les caractères non numériques avant conversion.

clean_location()

Sépare la ville du quartier, et corrige les noms de communes d'Abidjan selon une table de correspondance standardisée.

clean_title()

Normalise les espaces multiples et la casse des titres d'annonces.

extract_category_detail()

Classifie automatiquement les annonces en sous-catégories (robe, chaussure, boubou, pagne, sac, accessoire, veste, autre) à partir des mots-clés du titre.

4.4 Résultats du scraping

Critère / Composant	Statut
Items collectés	500 annonces
Items après nettoyage	500 items propres
Prix moyen	13 147,46 FCFA
Sous-catégorie principale	Autre (353) — Accessoire (56) — Sac (42)
Catégories secondaires	Chaussure (35) — Veste (14)
Format de sortie	raw_data.json + clean_data.json

4.5 Tests pytest (27 tests 100% réussis)

27 tests unitaires ont été écrits et passent tous avec succès :

- TestCleanPrice (7 tests) : prix normal, grand nombre, à débattre, gratuit, None, vide, euros
- TestCleanLocation (4 tests) : ville+quartier, ville seule, commune Abidjan, None
- TestCleanTitle (3 tests) : titre normal, espaces multiples, None
- TestExtractCategory (5 tests) : chaussure, robe, boubou, pagne, autre
- TestCleanData (4 tests) : doublons, colonnes, prix numérique, sous-catégorie
- TestSpider (4 tests) : extract_item, parse_listing, get_page succès, get_page erreur HTTP

Résultat : 27 passed in 5.13s Jalon 1 validé à 100%

5. Étape 2 API Flask + Base de données PostgreSQL

5.1 Modèles SQLAlchemy (models.py)

Les modèles de données ont été définis avec SQLAlchemy pour une persistance en base PostgreSQL. La table principale FashionItem contient tous les champs extraits lors du scraping, enrichis des champs calculés lors du nettoyage (prix numérique, devise, ville, quartier, sous-catégorie).

5.2 Endpoints REST + Swagger (routes.py)

L'API expose 7 endpoints documentés via Flask-RESTX (Swagger automatique) :

Critère / Composant	Statut
GET /health	Vérification de l'état de l'API
GET /api/data/	Liste paginée des annonces (filtres : ville, catégorie, prix min/max)
GET /api/data/{id}	Détail d'une annonce par identifiant
GET /api/stats/	Statistiques agrégées (prix moyen, répartition par catégorie)
GET /api/categories/	Liste des sous-catégories disponibles
POST /api/scrape	Déclenchement asynchrone d'un scraping via Celery
GET /api/tasks/{id}	Suivi du statut d'une tâche de scraping asynchrone

5.3 Documentation Swagger

La documentation interactive Swagger est accessible à l'adresse http://localhost:5000/api/docs et permet de tester tous les endpoints directement depuis le navigateur.

6. Étape 3 Dockerisation

6.1 Dockerfile

Le Dockerfile utilise l'image python:3.11-slim comme base et installe toutes les dépendances via pip. La construction est optimisée pour minimiser la taille de l'image finale.

6.2 docker-compose.yml

Le fichier docker-compose.yml orchestre 7 services interconnectés. La configuration utilise des volumes persistants pour PostgreSQL et des variables d'environnement définies dans le fichier .env.

6.3 Difficultés liées à Docker

La mise en place de Docker a rencontré plusieurs obstacles techniques :

- Docker Desktop refusait de démarrer à cause de la virtualisation désactivée dans le BIOS
- La mise à jour WSL (wsl --update) a échoué, nécessitant un téléchargement manuel du package
- Après activation de la virtualisation hardware, Docker Desktop a pu démarrer normalement
- L'erreur 'flask db upgrade — dossier migrations introuvable' a été résolue en remplaçant les migrations par db.create_all() dans run.py

7. Étape 4 Celery et automatisation

7.1 Configuration Celery (celery_app.py)

Celery est configuré avec Redis comme broker de messages. La tâche scrape_jiji est enregistrée et peut être déclenchée de manière asynchrone via l'endpoint POST /api/scrape.

7.2 Planification automatique Celery Beat

Celery Beat est configuré pour déclencher automatiquement un scraping complet toutes les 6 heures. Ce cycle permet de maintenir la base de données à jour avec les dernières annonces publiées sur Jiji.co.ci.

7.3 Suivi des tâches

Chaque tâche de scraping lancée de manière asynchrone reçoit un identifiant unique. L'endpoint GET /api/tasks/{id} permet de suivre l'état d'avancement (PENDING, STARTED, SUCCESS, FAILURE) de chaque tâche.

8. Étape 5 Monitoring (Prometheus + Grafana)

8.1 Métriques Prometheus

L'API Flask expose un endpoint /metrics compatible Prometheus via la bibliothèque prometheus-flask-exporter. Les métriques collectées sont :

- jiji_items_scraped_total — nombre total d'annonces scrapées
- jiji_scrape_total{status} — nombre de scrapings effectués (succès / échec)
- jiji_api_requests_total — nombre total de requêtes HTTP reçues par l'API
- Métriques HTTP standards (latence, code de réponse) générées automatiquement

8.2 Dashboard Grafana

Un dashboard nommé 'Jiji Mode CI — Monitoring' a été créé dans Grafana (http://localhost:3000) avec 3 panels :

- Panel 1 : Annonces scrapées — métrique jiji_items_scraped_total
- Panel 2 : Requêtes API totales — métrique jiji_api_requests_total
- Panel 3 : Scrapings effectués — métrique jiji_scrape_total

Prometheus est configuré en source de données dans Grafana avec l'URL interne http://prometheus:9090. La connexion a été validée avec le message 'Successfully queried the Prometheus API'.

9. Résultats et bilan final

L'ensemble des composants du niveau Or ont été livrés et validés. Voici le bilan complet :

Critère / Composant	Statut
Scraping Jiji.co.ci (500 items)	✅Validé — 4 catégories, 500 annonces
Nettoyage pandas	✅Validé — 500 items propres, prix moyen 13 147 FCFA
27 tests pytest	✅Validé — 27 passed in 5.13s
API Flask + Swagger	✅Validé — 7 endpoints documentés
Base de données PostgreSQL	✅Validé — tables créées via SQLAlchemy
Docker Compose (7 services)	✅Validé — tous les containers opérationnels
Celery Worker + Beat (6h)	✅Validé — scraping automatique configuré
Prometheus /metrics	✅Validé — endpoint 200, métriques exposées
Dashboard Grafana	✅Validé — 3 panels, source Prometheus connectée
Dépôt GitHub	✅Validé — code pushé, collaborateur invité

9.1 Statistiques clés des données collectées

Critère / Composant	Statut
Volume de données	500 annonces mode
Prix moyen	13 147,46 FCFA
Fourchette de prix	0 FCFA (gratuit) à plusieurs dizaines de milliers FCFA
Catégorie dominante	Autre (353 / 500 = 70,6%)
Accessoires	56 items (11,2%)
Sacs	42 items (8,4%)
Chaussures	35 items (7,0%)
Vestes	14 items (2,8%)
Villes représentées	Abidjan (majoritaire), San-Pédro, autres villes CI

10. Difficultés rencontrées et solutions apportées

Critère / Composant	Statut
UnicodeEncodeError dans les headers HTTP	Suppression du tiret long (em dash) — remplacé par -
URL /ci/fashion incorrecte (404)	Identification de la vraie URL : /fashion-and-beauty
Blocage HTTP 403 du serveur	Utilisation d'un User-Agent navigateur réel (Chrome)
Nettoyage : seulement 1 item retenu sur 100	Bug pandas sur drop_duplicates avec valeurs null — séparation des lignes avec/sans link
Prix incorrects (16.0 au lieu de 16 000)	Correction de la regex : suppression de tous les non-chiffres avant conversion float
Docker Desktop ne démarrait pas	Activation de la virtualisation Intel VT-x dans le BIOS
wsl --update échoue	Téléchargement manuel du package WSL2 depuis le lien officiel Microsoft
flask db upgrade : dossier migrations manquant	Remplacement par db.create_all() dans run.py
TLS handshake timeout lors du docker-compose --build	Utilisation du cache Docker existant sans --no-cache
/metrics retourne 404	Correction de la configuration prometheus-flask-exporter dans app.py

Conclusion

Ce projet a permis de construire une pipeline complète de collecte, traitement et exposition de données sur le marché de la mode en Côte d'Ivoire, depuis le scraping jusqu'au monitoring en production.

Les 500 annonces collectées sur Jiji.co.ci révèlent un marché vestimentaire ivoirien dynamique avec un prix moyen de 13 147 FCFA. Les accessoires, sacs et chaussures représentent les sous-catégories les plus actives après la catégorie générique. Abidjan concentre la très grande majorité des annonces, avec quelques vendeurs localisés dans d'autres villes comme San-Pédro.

D'un point de vue technique, ce projet a été l'occasion de mettre en pratique l'ensemble de la chaîne Data Engineering : scraping éthique, nettoyage avec pandas, API REST documentée, tâches asynchrones avec Celery, containerisation Docker et monitoring Prometheus/Grafana. Tous les composants du niveau Or ont été livrés et validés.

Les perspectives d'amélioration incluent l'enrichissement de la classification des sous-catégories (actuellement 70% en 'autre'), l'extraction de la date de publication des annonces (actuellement null), et l'ajout d'analyses statistiques plus poussées via des endpoints dédiés.

— Fin du rapport —